SimplArchive

Messenger — Module Manual

Conversations between the people of one archive,
sealed on their devices to keys only they hold

Generated from source

© 2026

Hebel Consulting GmbH
Schweighofplatz 7
6010 Kriens (LU)
Switzerland
support@simplarchive.dev

Contents

1 Introduction 1
2 Setting it up 2
3 Conversations 3
4 Messages 4
5 Photos and files 5
6 Backing up and restoring 6
7 What the server can see 7
8 The format 8
9 Not yet 9

1 Introduction

The **Messenger** lets the people of one archive talk to each other: one-to-one, or in a named group, with messages, photos and files. It is a **closed user group**: only users of the same tenant take part, and only those with an enrolled key.

Everything a member writes or sends is **sealed on their own device** to the keys of the conversation's members, and opened only on a member's device. The server stores and forwards what it cannot read.

The Messenger is a **free, open-source module**: it needs no licence, and a tenant administrator activates it for the tenant. Its clients are the SimplArchive apps: the phone app's Messages tile comes first (in preparation), the desktop client next. The web client stays out, because a browser cannot reach a hardware key.

1.1 Who can take part

A member is a user of the tenant with **at least one enrolled certificate** whose key they hold — a smart card, a security key, a phone's key. Nothing can be sealed to someone without one, so they cannot be added, and the people picker does not offer them.

While the Messenger is active, the archive **offers certificate enrolment** to its users, and the certificates they enrol are the ones conversations are sealed to.

2 Setting it up

1. **Activate the module** for the tenant — no licence is needed, and it never lapses. This repository's standalone start (`demo/up.sh`) activates it by itself.
2. **Choose where conversations are filed.** The module's setting *Where conversations are filed* takes the id of a repository or folder. Group conversations and the shared folders of one-to-one chats are created there. Give it an administrator's rights only: each member gets access to their own conversations and nothing else, and the Messenger keeps those rights in step with each conversation's members.
3. **Let people enrol.** Members need an enrolled certificate before anyone can start a conversation with them.

```
saconsole module activate --module messenger
echo <repository id> | saconsole module settings set messenger messenger.repositoryId --stdin
```

Every user's personal space gets a folder **My Messages**, provisioned by the archive while the module is active. The module may act only on that folder of the personal space, never on the rest of it.

3 Conversations

3.1 One-to-one

There is **one** conversation per pair of people, created the first time either starts it. Its shared folder sits in the configured repository; in each person's **My Messages** a folder named after the other person leads to it.

3.2 Groups

A **group** has a name and starts with its creator as its **administrator**. Administrators add members, remove them, and make other members administrators or take that back. Anyone may leave.

A group always keeps **at least one administrator**: the last one cannot step down, nor leave while others remain, until they have made someone else one. The administrators are listed on the group's folder, so the archive shows them too.

When the last member leaves, the conversation is removed, its folder and files with it.

3.3 Keys and epochs

Each conversation has a **conversation key**, and a new one for every **epoch**. A member holds the key of an epoch as a copy wrapped to their certificates.

- **Membership is the current epoch's keys.** Whoever holds a key for the current epoch is a member; the folder's access list follows.
- **Removing someone starts a new epoch.** An administrator's device makes a new key and wraps it to the members who remain, so the removed member cannot read what follows.
- **Leaving does too — made by someone who stays.** The person leaving makes no key, because they would then know it. Until a remaining member's app publishes the next epoch, nothing more is sent; the app does that by itself before its next message.
- **Adding someone starts a new epoch as well.** The administrator's device makes a new key for everyone, the newcomer included, who therefore reads from joining on and holds nothing that opens the history before it — nor is it listed to them.

The apps say both plainly when it happens.

4 Messages

- **Send.** The device seals the message under the current epoch's key before it leaves; the server refuses a message sealed under an older epoch, so a member who missed a removal fetches the new key first.
- **Edit.** The sender replaces the text; the message shows as edited. Only the sender can, and only if nobody changed it since they read it.
- **Delete for everyone.** The sender removes the text; the message's place in the thread stays, marked deleted.
- **Delete for me.** Hides the message for you only.
- **Read markers** only move forward, so an older device catching up changes nothing.

4.1 Read receipts

Beneath your latest message that somebody has read, the thread shows **"Read"** in a one-to-one chat, or **"Read by 2 of 4"** in a group; tapping that message and choosing **Read by…** lists who. A receipt is how far each member has read the conversation, not a mark per message, and it is always on: the same marker that counts your unread messages and clears the notification is what the others see.

4.2 The "New messages" notification

When a message arrives, every other member gets a notification that says **"New messages"** — never the sender, never the text, because it can reach a lock screen. It is **never mailed**. While it is unread, further messages fold into it rather than adding more, so the phone is pushed once per burst; reading the conversation clears it, and the next message pushes again.

The push itself carries nothing but the notification's id; the phone fetches the rest from the archive. Tapping it opens that conversation, the key dialog first if the key is needed.

5 Photos and files

A photo or file is sealed on the device like a message and uploaded straight to the archive's storage. It is filed once, as a **Message Object** in the conversation's folder, and each member finds it through their own chat folder.

- **Only sealed content is accepted.** The archive checks that every Message Object is a sealed envelope and refuses anything else, and hands sealed files out on every encryption tier.
- **Kept apart from the archive.** Message Objects live in a storage area of their own and are left out of search, previews, text recognition, retention schedules and legal holds: the archive cannot read them, and their lifetime is the conversation's.
- **Delete for me** removes your reference; **delete for everyone** removes the file for all.

6 Backing up and restoring

A **backup** holds all of your conversations in one file, **sealed only**: the messages as they were delivered, your own sealed copy of each conversation key, and each file as the envelope it was sent as. Nothing is decrypted to make it, so it opens only with a key whose certificate it was sealed to.

```
saconsole messenger backup --out conversations.simplarchive-messages
saconsole messenger restore --file conversations.simplarchive-messages
```

- **Restoring on another installation.** Each conversation comes back as **history**, marked **Restored** and read-only: you read it with your own key, nothing more is sent into it. Its messages carry a small **restored** mark, because the format has no sender signatures — they are as trustworthy as whoever restored them.
- **Only your own.** A backup is restored by the person it belongs to: the installation checks that its owner's e-mail is yours.
- **Twice changes nothing.** Messages keep their ids; what is already there is skipped. A second member restoring the same conversation joins it rather than making another.
- **Checked first.** Every entry is checked against the backup's own list of hashes before anything is restored; a changed or damaged backup is refused as a whole.
- **Continue here.** When everyone from a restored conversation is on this installation with an enrolled key (matched by e-mail), its thread offers **Continue here**. A one-to-one chat continues as the chat with that person — the one already started here, if there is one — and a group as a new group of the same people, administered by whoever continued it, with a new key for everyone. The restored history then sits above the live conversation; members who did not restore it see the conversation from there on, never history their own keys did not bring. Where somebody is missing, the thread names them and the conversation stays history.
- **Bound to your keys at the time.** A key enrolled only later cannot open older content; a renewed certificate on the same key can. A second enrolled key — a spare card — is your insurance.

7 What the server can see

The server sees	The server cannot see
who is in which conversation, and who administers a group	the text of any message
who sends when, and how large each message and file is	the content of any photo or file
read markers, edits and deletions — that they happened	any conversation key
a group's name, and the names on chat folders	

No forward secrecy in this version. Messages are sealed to the members' certificates, as sealed documents are. Someone who obtains a member's private key **and** the stored messages can read that member's past conversations. Keep keys on hardware that does not let them out.

8 The format

For those who audit it:

- A random 256-bit **conversation key** per epoch, wrapped by a sealed envelope to **every** certificate of every member — a member with several cards or devices opens it with any of them.
- Messages, photos and files are encrypted with that key using AES-256-GCM, with the format version, the conversation, the epoch, the message id and the sender bound into the authentication, so a message cannot be replayed into another conversation, epoch or sender.
- The client library and the key library are open source, in the Messenger's public repository.

9 Not yet

- **Restoring conversations you have left** is in preparation.
- **Disappearing messages** are not offered.

administrator 3
backup 6
closed user group 1
conversation key 3
enrolled certificate 1
epoch 3
forward secrecy 7
group 3
Message Object 5
Messenger 1
My Messages 2
read marker 4
repository
  setting 2
sealed 1